

Day 1 Notes: Verilog Fundamentals

1. What is Verilog?

Verilog is a Hardware Description Language (HDL), NOT a programming language.

Key Differences:

Programming Language	Hardware Description Language
Describes sequential instructions	Describes hardware structures
Executes in time	Exists in space (concurrent)
Runs on a processor	Synthesizes to gates/flip-flops

When you write Verilog, you're describing:

- Physical wires
- Logic gates (AND, OR, NOT, etc.)
- Registers (flip-flops)
- Multiplexers, decoders, etc.

Your code doesn't "run" — it gets:

1. **Simulated** (to verify behavior)
2. **Synthesized** (to actual hardware: FPGA/ASIC)

2. Where Verilog is Used

- **ASIC Design** - Custom chips (CPUs, GPUs, phone chips)
- **FPGA Programming** - Reconfigurable hardware
- **Digital Verification** - Testing designs before fabrication

3. Module Structure

A **module** is the fundamental building block of Verilog.

Basic Template:

```
module module_name(  
    input wire a,  
    input wire b,  
    output wire y  
);  
    // Internal logic goes here  
endmodule
```

Components:

1. **module/endmodule** - Defines boundaries
 2. **Ports** - Interface to outside world
 - **input** - Signals coming in
 - **output** - Signals going out
 - **inout** - Bidirectional (rare)
 3. **Port types:**
 - **wire** - For combinational logic (default)
 - **reg** - For procedural assignments (Day 2)
-

4. Port Declaration Styles

Old Style (Verilog-1995) ❌ Avoid:

```
module and_gate(a, b, y);  
    input a, b;  
    output y;  
endmodule
```

Modern Style (Verilog-2001+) ✅ Use This:

```
module and_gate(  
    input wire a,  
    input wire b,  
    output wire y  
);  
endmodule
```

Why modern is better:

- Direction and type in one place
 - Easier to read
 - Industry standard
-

5. Continuous Assignment

Used for combinational logic (output depends only on current inputs).

Syntax:

```
assign output_signal = expression;
```

Examples:

```
// AND gate  
assign y = a & b;
```

```
// OR gate  
assign y = a | b;
```

```
// XOR gate
assign y = a ^ b;

// NOT gate
assign y = ~a;

// Complex expression
assign y = (a & b) | (c & d);
```

Key Points:

- `assign` is **continuous** - output updates whenever inputs change
 - Describes **wires connecting gates**
 - Always used with `wire` type outputs
-

6. Verilog Operators

Bitwise Operators:

```
&    // AND
|    // OR
^    // XOR
~    // NOT
```

Example:

```
assign y = a & b & c; // 3-input AND
assign y = a | b;    // 2-input OR
assign y = ~a;       // Inverter
```

7. Testbenches

A testbench verifies your design through simulation.

Purpose:

- Apply input stimulus
- Observe output responses
- Verify correctness
- Debug before synthesis

Key Points:

- **NOT synthesized** (simulation only)
 - No ports (top-level module)
 - Uses system tasks (start with \$)
-

8. Testbench Structure

Template:

```
module tb_module_name;

    // 1. Declare signals
    reg input_signal;    // Use 'reg' for inputs you control
    wire output_signal; // Use 'wire' for outputs you observe

    // 2. Instantiate Design Under Test (DUT)
    module_name dut(
        .port_name(signal_name),
        .input_port(input_signal),
        .output_port(output_signal)
    );

    // 3. Apply stimulus
    initial begin
        // Setup waveform dumping
        $dumpfile("waveform.vcd");
        $dumpvars(0, tb_module_name);

        // Test cases
        input_signal = 0; #10; // Set value, wait 10 time units
        input_signal = 1; #10;

        // End simulation
        $finish;
    end
endmodule
```

9. reg vs wire in Testbenches

Type	Usage	Example
reg	Signals you assign (inputs to DUT)	reg a, b;
wire	Signals you observe (outputs from DUT)	wire y;

Note: reg in testbenches doesn't mean "register/flip-flop" — it's just a variable you can assign.

10. System Tasks

System tasks control simulation (start with \$):

Essential Tasks:

```
// Waveform generation
$dumpfile("filename.vcd"); // Create waveform file
$dumpvars(0, module_name); // Record all signals

// Display/Debug
$display("format", vars); // Print once (like printf)
$monitor("format", vars); // Print on any change
```

```
// Timing
$time                                // Current simulation time

// Control
$finish;                             // End simulation
```

Format Specifiers:

```
%b // Binary
%d // Decimal
%h // Hexadecimal
%t // Time
```

Example:

```
$display("Time=%0t a=%b b=%b y=%b", $time, a, b, y);
// Output: Time=10 a=0 b=1 y=1
```

11. The initial Block

```
initial begin
    // Executes once at time 0
    // Used for stimulus generation
end
```

Key points:

- Simulation-only (doesn't synthesize)
- Executes statements sequentially
- Used in testbenches for test patterns

12. Time Control

```
#delay // Wait specified time units
```

Example:

```
a = 0; b = 0; #10; // Set signals, wait 10 units
a = 1; b = 1; #20; // Set signals, wait 20 units
```

Time units are arbitrary (specify with ``timescale` directive - covered later).

13. Module Instantiation

Connecting a module in another module or testbench:

Syntax:

```
module_name instance_name(
    .port_name(signal_name),
```

```
        .port_name(signal_name)
    );
```

Example:

```
// Design
module and_gate(
    input wire a,
    input wire b,
    output wire y
);
    assign y = a & b;
endmodule

// Testbench instantiation
and_gate dut(
    .a(a),          // Connect testbench signal 'a' to module port 'a'
    .b(b),
    .y(y)
);
```

dut = "Design Under Test" (common naming convention)

14. Complete Example: OR Gate

Design File: or_gate.v

```
module or_gate(
    input wire a,
    input wire b,
    output wire y
);
    assign y = a | b;
endmodule
```

Testbench File: tb_or_gate.v

```
module tb_or_gate;
    reg a, b;
    wire y;

    or_gate dut(
        .a(a),
        .b(b),
        .y(y)
    );

    initial begin
        $dumpfile("or_gate.vcd");
        $dumpvars(0, tb_or_gate);

        $display("time | a | b | y");
        $display("-----|---|---|---");

        a = 0; b = 0; #10;
        $display("%4t | %b | %b | %b", $time, a, b, y);

        a = 0; b = 1; #10;
```

```

        $display("%4t | %b | %b | %b", $time, a, b, y);

a = 1; b = 0; #10;
$display("%4t | %b | %b | %b", $time, a, b, y);

a = 1; b = 1; #10;
$display("%4t | %b | %b | %b", $time, a, b, y);

    $finish;
end
endmodule

```

15. Simulation Workflow

```

# 1. Compile design and testbench
iverilog -o sim design.v testbench.v

# 2. Run simulation
vvp sim

# 3. View waveforms
gtkwave waveform.vcd

```

16. File Organization Best Practices

```

project/
├── or_gate.v      ← Design (synthesizable)
└── tb_or_gate.v  ← Testbench (simulation only)

```

Always separate:

- Design files (RTL code)
- Testbench files (verification code)

Why:

- Clean organization
 - Synthesis tools only process design files
 - Industry standard practice
-

17. Common Mistakes to Avoid

Wrong:

```

// Semicolons in port list
module gate(
    input a;
    input b;
    output y;
);

```

✓ **Correct:**

```
// Commas in port list
module gate(
    input a,
    input b,
    output y
);
```

✗ **Wrong:**

```
// Using "or" as operator
assign y = a or b;
```

✓ **Correct:**

```
// Use symbolic operators
assign y = a | b;
```

✗ **Wrong:**

```
// Declaring unused ports
module gate(
    input a, b, c,
    output y
);
    assign y = a & b; // 'c' is unused!
endmodule
```

✓ **Correct:**

```
// Only declare what you use
module gate(
    input a, b,
    output y
);
    assign y = a & b;
endmodule
```

18. Truth Tables for Basic Gates

AND Gate:

a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate:

a	b	y
0	0	0
0	1	1
1	0	1
1	1	1

XOR Gate:

a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

19. Key Takeaways

1. Verilog describes hardware, not software
 2. Every signal is a wire or storage element
 3. **assign** creates combinational logic
 4. Testbenches verify before synthesis
 5. Follow specifications exactly
 6. Modern syntax > old syntax
 7. Separate design and testbench files
 8. Simulate before synthesizing
-

20. Professional Habits Learned

- ✓ Follow specifications exactly
 - ✓ Use modern coding style
 - ✓ Test every design thoroughly
 - ✓ Fix issues completely before moving on
 - ✓ Write readable, well-formatted code
 - ✓ Every port must have a purpose
 - ✓ Version control friendly structure
-

Tools Setup (Linux - Kubuntu)

```
# Install simulator
sudo apt install iverilog

# Install waveform viewer
sudo apt install gtkwave

# Install VS Code (optional)
```

```
sudo snap install code --classic
```

VS Code Extension:

- "Verilog-HDL/SystemVerilog" by mshr-h
-